

Nama: Jonathan Alvindo Fernandi  
NIM: 2602089143  
Kampus: Kemanggisan  
Departemen: Computer Science

1. Buatlah sebuah ilustrasi *class relationship* (*association*, *aggregation*, dan *composition*). Sebutkan dahulu tema yang kalian gunakan dan gambarkan *class* apa saja beserta garis *relationship*-nya. Pastikan ketiga *relationship* tersebut ada pada ilustrasi kalian.

Jawab:

a. *Association*

Tema:

*Association* antara Class Mahasiswa dengan Class MataKuliah, dan Class Dosen dengan Class MataKuliah



b. *Aggregation*

Tema:

*Aggregation* antara Class Mahasiswa dengan Class MataKuliah, dan Class Dosen dengan Class MataKuliah



c. *Composition*

Tema:

*Composition* antara Class Tujuan dengan Class MataKuliah, dan Class Materi dengan Class MataKuliah



2. Diberikan beberapa *class* sebagai berikut: Buatlah parent dari kedua *class* tersebut!

```

public class Cupboard {
    private String modelName;
    private Double length;
    private Double width;
    private Double price;
    private String ID;
}

public class Sofa {
    private String ID;
    private String modelName;
    private Double price;
    private String exterior;
}

```

3. Tuliskan *code* untuk *generate random integer* dari *range* (caranya bebas):
  - 1–9
  - 10–29
  - 50–(-10)
  - 0–(-99)
4. Sebutkan dan jelaskan minimal 3 perbedaan antara ArrayList dan Vector. Salah satu perbedaannya adalah Vector itu *synchronized*, sedangkan ArrayList tidak. Jelaskan maksud dari *synchronized* dan contoh kasusnya!

Jawaban:

- ArrayList tidak *synchronized* (beberapa *thread* dapat bekerja pada ArrayList pada saat yang bersamaan), sedangkan Vector *synchronized* (hanya satu *thread* pada satu waktu yang dapat mengakses *code*). Contoh kasusnya adalah sebuah *web application* yang memungkinkan banyak *user* untuk menambah dan menghapus *item* dari keranjang belanja yang sama pada saat yang bersamaan. Jika ArrayList digunakan untuk menyimpan *item* dalam keranjang, ada kemungkinan bahwa dua pengguna dapat mencoba menambah atau menghapus *item* secara bersamaan karena bersifat tidak *synchronized*. Jika Vector digunakan untuk menyimpan *item* dalam keranjang, setiap operasi penambahan atau penghapusan *item* akan menunggu operasi sebelumnya selesai karena bersifat *synchronized*.
- ArrayList meng-*increment* 50% dari ukuran *array* saat ini jika jumlah elemen melebihi kapasitasnya, sedangkan Vector meng-*increment* 100% dari ukuran *array* saat ini (menggandakan ukuran *array*) jika jumlah elemen melebihi kapasitasnya.
- ArrayList bukan sebuah *legacy class* (diperkenalkan pada JDK 1.2), sedangkan Vector merupakan sebuah *legacy class*.

Kasus

Patreon

There are many donation websites that enable us to support artists, indie game developers, musicians, etc. One of them is Patreon. Patreon Creators (the ones you donate to) can set various subscription/subscription levels divided by the amount of donation. The more you donate per month, the higher the level, and you usually gain early access to new works. While Patreon users can decide which creators they want to donate/subscribe to.

Now, let us try to create a smaller version of Patreon.

Imagine now that you are a Patreon Admin who is testing Patreon features. Like YouTube, each user can upload their own works/contents, and can support/subscribe to other users. Subscriptions have several levels, from lowest to highest: bronze, silver, and gold. Higher-level subscription can access lower level subscription while the opposite is not possible. For example, Gold can access Gold, Silver, and Bronze content, while Bronze is only able to access Bronze content but not Silver or Gold. Here are the details:

#### Main menu

1. Insert new patreon user
  - The program will ask for a username. Assume that all usernames are unique, so no validation is needed.
  - Insert the user data into a dynamic array. New users initially always have an empty list of work, subscribers, and supported creators.
2. Login
  - The program will display all available usernames.
  - Then it will ask you to enter your username to log in. Ensure the username entered exists (case-sensitive).
3. Exit
  - This will terminate the program.

#### User menu

This menu will be displayed after logging in.

1. Insert new work
  - The program will ask for a work type. The value is either music, video, or image.
  - If the work type is music, next it will ask for a genre name.
  - If the work type is video, next it will ask for a video duration.
  - If the work type is image, next it will ask for an image type. The value is either JPEG or PNG.
  - Then it will ask for a work title.
  - Then it will ask for the subscription level. The value is either bronze, silver, or gold.
  - A work ID in the format of 9 random numbers will be generated automatically.
  - Insert the work data into this user's work list.
2. Support a creator
  - The program will display all users/creators that this user hasn't subscribed to and ask the user to enter a username to subscribe.
  - Then it will ask for the subscription level. The value is either bronze, silver, or gold.
  - Add the entered data to this user's supported creators list.
3. View all subscriber
  - If there are no subscribers, display a message "No subscriber".

- Otherwise, the program will display all users subscribed to you, grouped by each subscription level.
4. View your work
    - If there are no works, display a message “No works uploaded yet”.
    - The program will display all this user’s inserted work data based on work type (music, video, or image).
  5. View accessible work
    - If there isn't any work accessible, display a message “No works you can see”.
    - The program displays all types of work data accessible by this user, grouped by subscription level.
  6. Log out
    - Selecting this will log the user out and return to the main menu.